

Reviewer Pack — Hyperplane Arrangement Region Count Bounds Karchmer-Wigderson...

Entry 4ae2da3d0aa7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 08:10:43 UTC

Hyperplane Arrangement Region Count Bounds Karchmer-Wigderson Communication Complexity

Entry ID: 4ae2da3d0aa7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 08:10:43 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Hyperplane Arrangements) **Field B** (complexity object): Communication Complexity of Karchmer-Wigderson Games

Statement:

For a boolean function f , the communication complexity of its Karchmer-Wigderson game is at most the number of regions in the hyperplane arrangement induced by the function's clauses.

Rationale (proposer's reasoning):

The regions in the hyperplane arrangement partition the input space, potentially corresponding to different communication protocols, and their count may capture the minimal protocol size.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 474e89abb485bd21

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            clauses.append(clause)
        return clauses

    def dpll(clauses, assignment=[]):
        if not clauses:
            return True
        unit_clause = next((c for c in clauses if len(c) == 1), None)
        if unit_clause:
            literal = unit_clause[0]
            var = abs(literal)
            if literal > 0:
                assignment[var - 1] = True
            else:
                assignment[var - 1] = False
            return dpll([c for c in clauses if literal not in c], assignment)
        pure_literal = next((i for i in range(1, len(assignment) + 1) if (i not in [abs(lit) for lit in clauses])), None)
        if pure_literal is not None:
            literal = pure_literal
            var = abs(literal)
            if literal > 0:
                assignment[var - 1] = True
            else:
                assignment[var - 1] = False
            return dpll([c for c in clauses if literal not in c], assignment)
        literal = random.choice(clauses[0])
        var = abs(literal)
        if literal > 0:
            assignment[var - 1] = True
        else:
            assignment[var - 1] = False
```

```

        return dpll([c for c in clauses if literal not in c], assignment) or dpll([c for c in clauses if literal in c], assignment)

def count_regions(clauses):
    n = len(clauses[0])
    regions = [set()]
    for clause in clauses:
        new_regions = []
        for region in regions:
            r1 = {x for x in region if any(lit > 0 and lit != x for lit in clause)}
            r2 = {x for x in region if all(lit < 0 or lit == x for lit in clause)}
            if len(r1) > 0:
                new_regions.append(r1)
            if len(r2) > 0:
                new_regions.append(r2)
        regions = new_regions
    return sum(len(region) for region in regions)

n = random.randint(5, 40)
clauses = generate_cnf(n)
region_count = count_regions(clauses)
communication_complexity = dpll(clauses)

return {
    "metric_name": "communication_complexity",
    "metric_value": communication_complexity,
    "instances_tested": 1,
    "conjecture_holds": region_count >= communication_complexity,
    "counterexample": "" if region_count >= communication_complexity else f"Graph with n={n},
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 89))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_75854521.py", line 90, in <module>
    result = run_trial(seed)
             ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_75854521.py", line 75, in run_trial
    communication_complexity = dpll(courses)
                                ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_75854521.py", line 54, in dpll
    assignment[var - 1] = False
    ~~~~~
IndexError: list assignment index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before producing results, preventing evaluation of conjecture validity. | next: Fix the IndexError in test_75854521.py and re-run with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	103338
2	preregistration	ollama_remote	qwen3:8b	0	0	36447
3	novelty	ollama_remote	qwen3:8b	0	0	27223
4	novelty	ollama_remote	qwen3:8b	0	0	20304
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14652
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11154
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11839
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11159
9	judge	ollama_remote	qwen3:8b	0	0	20816

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 256933 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/4ae2da3d0aa7.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4ae2da3d0aa7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4ae2da3d0aa7.tar.gz` (if generated)